

Event Driven Architecture

Remote Controlled Car

Sujit Kumar Chakrabarti

1 Event-Driven Architecture

2 Example - Remote Controlled Car

- Interrupt-Driven Architecture
- Publish-Subscribe Architecture

3 Design Competition

Event Driven Architectures

Examples

- Embedded applications: car, lift, train etc.
- GUI applications
- Web services

Event Driven Architectures

Examples

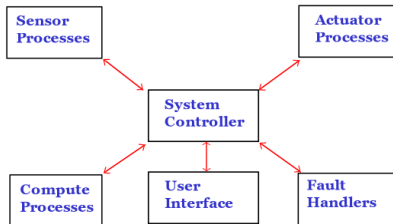
- Embedded applications: car, lift, train etc.
- GUI applications
- Web services

Event

- Something that happens in time
- May happen inside or outside the system
- May or may not carry additional data

Event Driven Architectures

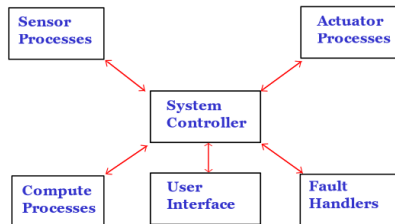
Variants – Interrupt driven model



- Useful for real-time systems
- Known number of interrupts/event
- Control passes to event/interrupt handler
- May be pre-emption based

Event Driven Architectures

Variants – Interrupt driven model



- Useful for real-time systems
- Known number of interrupts/event
- Control passes to event/interrupt handler
- May be pre-emption based

Advantages:

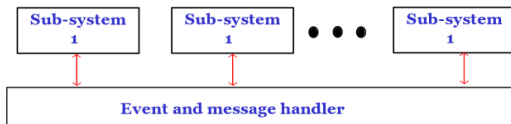
- Fast
- Simpler, safer, secure

Disadvantages:

- Rigid
- Hard to debug

Event Driven Architectures

Variants – Broadcast model



- Another name: publish-subscribe architecture
- Sub-systems may be distributed.
- Registration/subscription and broadcast/publish
- Control policy is distributed.

Event Driven Architectures

Variants – Broadcast model



- Another name: publish-subscribe architecture
- Sub-systems may be distributed.
- Registration/subscription and broadcast/publish
- Control policy is distributed.

Advantages:

- Modular, loosely coupled
- Extensible, fault tolerant

Disadvantages:

- Communication overhead
- Untracked messages
- Security
- Hard to debug

1 Event-Driven Architecture

2 Example - Remote Controlled Car

- Interrupt-Driven Architecture
- Publish-Subscribe Architecture

3 Design Competition

Remote-Controlled Toy Car



System Overview

The system consists of two physical units:

Remote Control Unit

- Sends control commands to the car.
- Supported commands:
 - MOVE
 - STOP

Robotic Car

- Receives commands from the remote control.
- Detects obstacles using a distance sensor.
- Executes all control logic using a central controller.

Hardware Architecture

Sensors

- Command Receiver
 - MOVE_CMD
 - STOP_CMD
- Distance Sensor
 - Active only while moving
 - Reports every 100 ms

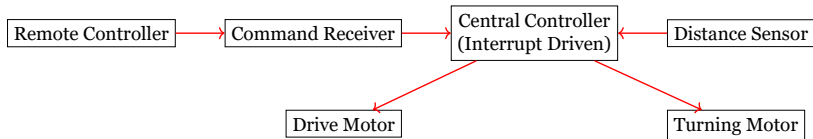
Actuators

- Drive Motor
 - FORWARD
 - STOP
- Turning Motor
 - TURN_RIGHT_90

Central Controller

Receives all interrupts, maintains system state, and issues actuator commands.

System Architecture



System States

STOPPED

- Drive motor stopped.
- Distance sensor inactive.

MOVING

- Moving forward.
- Distance sensor active.
- Obstacle reports every 100 ms.

Events

Event	Source
MOVE_CMD	Receiver
STOP_CMD	Receiver
OBSTACLE_DETECTED	Distance Sensor
CLEAR_PATH	Distance Sensor

State Transition: STOPPED State

Event: MOVE_CMD

- Activate distance sensor.
- Start forward motor.
- Next state: MOVING

Event: STOP_CMD

- No action.
- Remain in STOPPED

Sensor events are ignored because the sensor is inactive.

State Transition: MOVING State

Event: STOP_CMD

- Stop forward motor.
- Disable distance sensor.
- Next state: STOPPED

Event: MOVE_CMD

- No action.
- Remain in MOVING

Obstacle Handling

Event: OBSTACLE_DETECTED

Actions:

- 1 Stop forward motor.
- 2 Turn right by 90° .
- 3 Resume forward motion.

Next state:

MOVING

Event: CLEAR_PATH

- Continue moving forward.

Interrupt-Driven Control Logic

```
state := STOPPED

on interrupt(event)

    switch state

        STOPPED:

            if event == MOVE_CMD then
                sensor.enable()
                drive.forward()
                state := MOVING
            end if

        MOVING:
            ...
    end switch
```

Interrupt-Driven Control Logic

MOVING:

```
    if event == STOP_CMD then
        drive.stop()
        sensor.disable()
        state := STOPPED

    else if event == OBSTACLE_DETECTED then
        drive.stop()
        turn.right90()
        drive.forward()

    else if event == CLEAR_PATH then
        skip // continue forward
    end if
end switch
```

Timing Requirements

Distance Sensor

Sampling period:

100 ms

STOP Command

- Highest-priority interrupt.
- Vehicle must stop immediately.

Obstacle Detection

Turning must begin immediately after `OBSTACLE_DETECTED`.

Architectural Properties

- Centralised control.
- Interrupt-driven execution.
- Reactive behaviour.
- Finite-state control model.
- Periodic sensing while moving.
- Immediate response to STOP commands.

STOPPED $\longleftrightarrow$ MOVING

1 Event-Driven Architecture

2 Example - Remote Controlled Car

- Interrupt-Driven Architecture
- Publish-Subscribe Architecture

3 Design Competition

Motivation

The original architecture used a centralised interrupt-driven controller.

Alternative design:

Publish–Subscribe Architecture

Components communicate through an Event Bus.

- Publishers generate events.
- Subscribers consume events.
- Components are loosely coupled.

Goal: preserve system behaviour while improving extensibility.

System Requirements

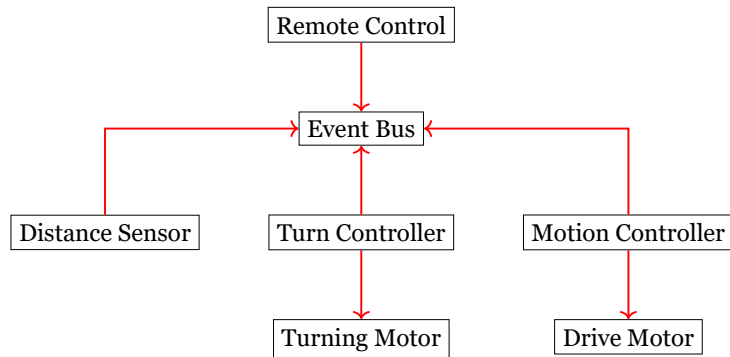
The remote control provides:

- MOVE
- STOP

The car shall:

- Move forward after receiving MOVE.
- Stop immediately after receiving STOP.
- Detect obstacles using a distance sensor.
- Turn right by 90° when an obstacle is detected.
- Continue moving after completing the turn.

Architectural Overview



Architectural Components

Event Bus

- Event publication
- Event subscription
- Event delivery

Publishers

- Remote Control Receiver
- Distance Sensor

Subscribers

- Motion Controller
- Turn Controller

Remote Receiver and Distance Sensor

Remote Receiver

Publishes:

- MOVE_CMD
- STOP_CMD

Distance Sensor

- Active only while moving.
- Generates events every 100 ms.

Publishes:

- OBSTACLE_DETECTED
- CLEAR_PATH

Controllers

Motion Controller

Subscribes to:

- MOVE_CMD
- STOP_CMD
- TURN_COMPLETED

Responsibilities:

- Manage motion state
- Start/stop motor
- Enable sensor

Turn Controller

Subscribes to:

- OBSTACLE_DETECTED
- STOP_CMD

Responsibilities:

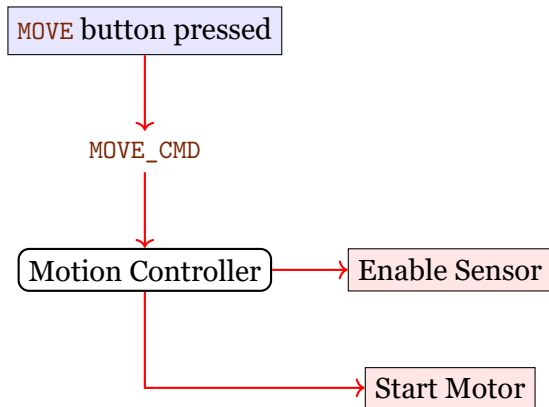
- Perform right turn
- Abort turn if needed
- Publish completion event

Events

Events

Event	Publisher
MOVE_CMD	Remote Receiver
STOP_CMD	Remote Receiver
OBSTACLE_DETECTED	Distance Sensor
CLEAR_PATH	Distance Sensor
TURN_COMPLETED	Turn Controller

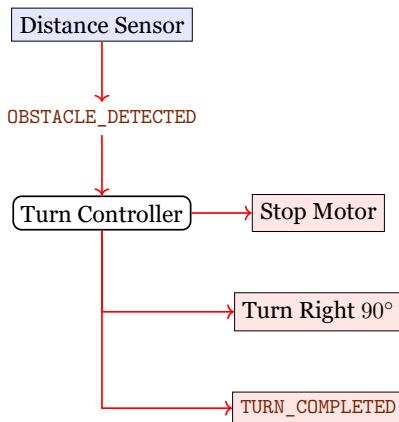
Scenario 1: Start Moving



Result:

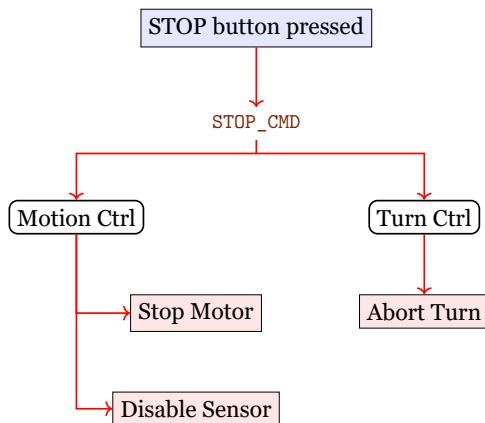
STOPPED → *MOVING*

Scenario 2: Obstacle Avoidance



Motion Controller receives **TURN_COMPLETED** and resumes forward movement.

Scenario 3: Emergency Stop



Multiple subscribers react independently to the same event.

Topic-Based View

Possible Event Bus topics:

Topic	Publisher
/command/move	Remote Receiver
/command/stop	Remote Receiver
/sensor/obstacle	Distance Sensor
/sensor/clear	Distance Sensor
/turn/completed	Turn Controller

Subscribers register interest in topics rather than specific components.

Distributed State Management

Motion Controller

States:

- STOPPED
- MOVING

Turn Controller

States:

- IDLE
- TURNING

Unlike the centralised architecture, state is distributed among multiple components.

Timing Requirements

Distance Sensor

Publish status every

100 ms

while moving.

STOP Command

Highest-priority event.

- Stop motor immediately.
- Disable sensor.
- Abort ongoing turn.

Advantages and Disadvantages

Advantages

- Loose coupling
- Extensible
- Multiple subscribers
- Easier feature addition
- Supports telemetry/logging

Disadvantages

- More complex
- Harder debugging
- Event ordering issues
- Additional overhead

Comparison

Centralised Controller	Publish-Subscribe
Single controller	Distributed control
Tight coupling	Loose coupling
Simple	Extensible
Easy verification	Easy evolution
Small embedded systems	Large robotic systems

Conclusion

- Both architectures satisfy the same requirements.
- Publish–subscribe provides better modularity.
- Components communicate through events rather than direct calls.
- Suitable when future extensions are expected.

Event Bus = Decoupled Communication

Questions?

Event-Driven Systems

Activity

Lift control system

- Events: Request - floor, car
- Sensors: Floor sensor, door sensor, obstruction sensor
- Actuator: Car motor, door motor
- ...

Event-Driven Systems

Activity

Lift control system

- Events: Request - floor, car
- Sensors: Floor sensor, door sensor, obstruction sensor
- Actuator: Car motor, door motor
- ...

Design challenge:

- Teams of 3.
- **Architectural design.** Each team will come up with a architecture diagram + justification.
- **Debate.** For some of the applications, two teams working on it will be called upon to present their work, and if required debate in favour of their decisions.